IS Audit & Control

Abdul-Rahman Mahmood
Assistant Professor, Computer Science, FAST-NU

abdulrahman@nu.edu.pk
alphapeeler.sf.net/pubkeys/pkey.htm
pk.linkedin.com/in/armahmood
bqb-tsid-asp
alphapeeler
alphapeeler
armahmood786
http://alphapeeler.sf.net/
reddit.com/user/alphapeeler
www.flickr.com/alphapeeler
http://alphapeeler.tumblr.com
armahmood786@jabber.org
alphapeeler@aim.com
abdulmahmood-sss
alphapeeler@icloud.com
pinterest.com/alphapeeler

Auditing Web Servers and Web Applications

Web Auditing Components

- **Web Audit Component Key Concerns**

Web Audit Component	Key Concerns
Web platform	Security of the operating system, physical and network protection to the host
Web server	Default settings, sample code, general misconfigurations, logging
Web application	Development framework security settings, default application settings, input validation, incorrectly serving up data, access to company confidential data, general misconfigurations

Table 8-1 Web Auditing Components

Test Steps for Auditing Web Servers

- **1. Verify that the web server is running on a dedicated system and not in conjunction with other critical applications.**
- **2. Verify that the web server is fully patched and updated with the latest approved code.**
- **3. Verify that unnecessary services, modules, objects, and APIs are removed or disabled. Running services and modules should be operating under the least privileged accounts.**
  - FTP, SMTP, Telnet, extra server extensions, and NNTP should be disabled if they are not required.
  - Use netstat or a more robust process to port-mapping utility.

Test Steps for Auditing Web Servers

- **4. Verify that only appropriate protocols and ports are allowed to access the web server.**
  - TCP/IP stack on the server should be hardened to allow only appropriate protocols.
  - NetBIOS and Server Message Block (SMB) should be disabled on IIS servers.
  - only TCP on ports 80 (HTTP) and 443 (SSL) should be allowed to access the web server.
  - review the negotiated ciphers allowed by Secure Sockets Layer (SSL) transactions.
- **5. Verify that accounts allowing access to the web server are managed appropriately and hardened with strong passwords.**

Test Steps for Auditing Web Servers

- **6. Ensure that appropriate controls exist for files, directories, and virtual directories.**
  - Website content
  - Website scripts
  - System files (such as %windir%\system32 or **wwwroot**.
  - Tools, utilities, and software development kits.
- **7. Ensure that the web server has appropriate logging enabled and secured.**
- **8. Ensure that script extensions are mapped appropriately.**
  - Scripts might allow an attacker to execute the code of his or her choice, potentially compromising the web server.
  - .idq, .htw, .ida, .shtml, .shtm, .stm, .idc, .htr, & .printer.
  - Script extensions not used are mapped to 404 web page handler or simply denied altogether.

Test Steps for Auditing Web Servers

- **9. Verify the validity and use of any server certificates in use.**
  - Verify with the help of the administrator that any certificates are used for their intended purpose and have not been revoked. Certificate data ranges, public key, and metadata all should be valid. If any of these have changed, consider the need for a new certificate that reflects your current needs.

Test Steps for Auditing Web Applications

- **1. Ensure that the web application is protected against injection attacks.**
  - in a SQL injection attack, SQL code is passed through the web interface, and the database is asked to perform functions out of bounds of your authorization.
    - Validate all input such as values, length, and character sets.
    - Perform a code review if possible for all calls to external resources.
    - Tools to find injection vulnerabilities (**www.acunetix.com**). **Burp Suite** from www.portswigger.net.
    - Consider hiring third-party help if the application is particularly sensitive, you lack the resources, or you need to verify items such as regulatory compliance.

Test Steps for Auditing Web Apps

- **2. Review the website for cross-site-scripting vulnerabilities.**
  - Cross-site scripting (XSS) allows the web application to transport an attack from one user to another end user's browser.
  - validate every header, cookie, query string, form field, and hidden field.
  - CIRT.net contains two tools, Nikto and a Nessus plug-in, that you might be able to use to help you partially automate the task of looking for XSS vulnerabilities on your web server.

Test Steps for Auditing Web Apps

- **3. Review the application for broken authentication and session management vulnerabilities.**
  - When a user enters an invalid credential into a login page, don't return which item was incorrect. Show a generic message instead such as, "Your login information was invalid!"
  - Never submit login information via a GET request. Always use POST.
  - Use SSL to protect login page delivery and credential transmission.
  - Remove dead code and client-side viewable comments from all pages.

Test Steps for Auditing Web Apps

- Do not depend on client-side validation. Validate input parameters for type and length on the server, using regular expressions or string functions.
- Use parameterized DB queries or stored procedures.
- DB connections should be created using a lower privileged account.
- Application shouldn't log into DB using sa/dbadmim.
- One way to store passwords is to hash passwords in a DB or flat file using SHA-256 or greater with a random SALT value for each password.
- Prompt user to close his browser to ensure that header authentication information has been flushed.
- Ensure that cookies have an expiration date, and do not store passwords in clear-text.
- www.owasp.org/index.php/Guide_to_Authentication.

Test Steps for Auditing Web Apps

- 4. Verify that proper object reference and authorization controls are enforced.
- 5. Verify that controls are in place to prevent Cross Site Request Forgery (CSRF or XSRF).
  - CSRF attacks exploit the trust a website has for the authenticated user. Attackers exploit this trust by sending embedded images, scripts, iframe elements, or other methods to call a command that executes on the web server while you are logged in with your credentials.
  - Example of this type of attack as an image request:
  - <img src="http://mybank.com/transfer?acct=mine&amt=100&to=attacker">
  - Use the methodology for uniquely creating tokens for each link.
  - OWASP provides tools for developers to create applications that securely create and manage unique tokens.

Test Steps for Auditing Web Apps

- 6. Review controls surrounding maintaining a secure configuration.
  - Check out OWASP's Open Software Assurance Maturity Model (SAMM) project online at www.owasp.org/index.php/SAMM.
- 7. Verify that secure cryptographic storage mechanisms are used correctly.
- 8. Verify that proper controls are in place to restrict URL filtering.
  - Appropriate filtering ensures that only authenticated users have access to each restricted page that they are authorized by their role to view.

Test Steps for Auditing Web Apps

- 9. Evaluate transport layer protection mechanisms (network traffic encryption) to protect sensitive information.
  - HTTP, FTP, and Telnet transmits info in cleartext.
  - Use SSL/TLS. Port-mapping tools can be used to monitor specific connections to the insecure web application from the client.
  - OpenSSL can also be used to validate available ciphers and versions.
- 10. Review the web application redirects and forwards to verify that only valid URLS are accessible.

Test Steps for Auditing Web Apps

- Using an unchecked redirect, attackers may be able to redirect users to the attacker's website using a URL that looks as if it comes from your domain:
- http://www.mydomain.com/redirect.asp?url=badsite.com
- unchecked forward :
- http://www.mydomain.com/somepage.asp?fwd=adminsite.jsp
- Automated scanners may be used to automatically scan and verify a website for the proper handling of redirects and forwards. Redirects and forwards are called transfers in.NET framework.

Test Steps for Auditing Web Apps

- 11. Verify that all input is validated prior to use by the web server.
  - Common items for positive filtering include:
  - Data type (e.g. string, integer, and real)
  - Allowed character set
  - Minimum and maximum length
  - Whether null is allowed
  - Whether the parameter is required or not
  - Whether duplicates are allowed
  - Numeric range
  - Specific legal values (e.g., enumeration)
  - Specific patterns (e.g., regular expressions)

Test Steps for Auditing Web Apps

- Tools & Technologies

Tool	Website
BackTrack	www.backtrack-linux.org
Burp Suite	http://portswigger.net/burp/
Samurai Web Testing Framework	http://samurai.inguardians.com
Web Sleuth	www.sandsprite.com/Sleuth
Paros Proxy	www.parosproxy.org
WebInspect	www.spidynamics.com/products/webinspect (redirects to HP)
Nikto	www.cirt.net/nikto2
XSS plug-in for Nessus	www.nessus.org/plugins/index.php?view=single&id=39466
Apache JMeter	http://jakarta.apache.org/jmeter
Google Skipfish	http://code.google.com/p/skipfish

Test Steps for Auditing Web Apps

- Knowledge Base

Website	Website
Apache website	www.apache.org
Microsoft IIS website	www.microsoft.com/windowsserver2003/iis
IIS answers	www.iisanswers.com
UrlScan tool for IIS	http://technet.microsoft.com/en-us/security/cc242650.aspx
Open Web Application Security Project (OWASP)	www.owasp.org
OWASP Application Security Verification Standard	www.owasp.org/index.php/OWASP_Application_Security_Verification_Standard_(ASVS)
Web Application Security Consortium	www.webappsec.org
Common Weakness Enumeration (CWE)	cwe.mitre.org
CGI Security	www.cgisecurity.net
Security Guidance for IIS	http://technet.microsoft.com/en-us/library/dd450371(WS.10).aspx
IIS Lockdown Tool	http://technet.microsoft.com/en-us/library/dd450372(WS.10).aspx
Google Code University—Web Security	http://code.google.com/edu/security